

Minimizing False Positives in Static Bug Detection via LLM-Enhanced Path Feasibility Analysis

Xueying Du¹ Kai Yu¹ Chong Wang² Yi Zou¹ Wentai Deng¹
 Zuoyu Ou¹ Xin Peng¹ Lingming Zhang³ Yiling Lou¹

¹Fudan University, China

²Nanyang Technological University, Singapore

³University of Illinois Urbana-Champaign, USA

Abstract

Static bug analyzers play a crucial role in ensuring software quality. However, existing analyzers for bug detection in large codebases often suffer from high false positive rates. This is primarily due to the limited capabilities of analyzers in path feasibility validation with multiple conditional branches and complex data dependencies. While current LLM-based approaches attempt to address this issue, their effectiveness remains limited due to insufficient constraint cascade analysis and scalability challenges in large projects. To address this challenge, we propose an iterative path feasibility analysis framework LLM4PFA. By leveraging LLM agent based targeted constraint reasoning, and key context-aware analysis driven by agent planning, LLM4PFA effectively enhances complex inter-procedural path feasibility analysis for minimizing false positives in static bug detection. Evaluation results show that LLM4PFA precisely filters out 72% to 96% false positives reported during static bug detection, significantly outperforming all the baselines by 41.1% - 105.7% improvements; meanwhile LLM4PFA only misses 3 real bugs of 45 true positives.

1 Introduction

Static bug analyzers play a crucial role in ensuring software security, reliability, and maintainability. They help identify potential bugs early in the development process by scanning the codebase without extensive program execution (Cai et al., 2023, 2022, 2021; Shi et al., 2020, 2018; Sui et al., 2012; Vassallo et al., 2020; Yan et al., 2018). Popular analyzers like GitHub CodeQL (cod, 2025), Facebook Infer (inf, 2025), and CppCheck (cpp, 2024) are widely used for industrial bug detection. Their analysis of many bug types (e.g., null pointer dereference and use-after-free) typically follows a standardized approach: identifying sources (i.e., potential origins of issues) and sinks (i.e., locations

where issues manifest) based on predefined rules, then checking whether a feasible execution path exists from sources to sinks. If such a path is found, the tools flag a potential bug.

However, in complex software systems, sources and sinks often reside in different functions. In cases where function call chains between a source and a sink are lengthy and involve multiple conditional branches and complex data dependencies, traditional static analyzers struggle to comprehensively validate path feasibility between sources and sinks, resulting in high false positive rates. For instance, our manual annotation of sampled bug reports reveals that popular tools produce an extremely high rate (e.g., over 90%) of false positives for use-after-free bugs across large-scale software such as Linux Kernel. This significantly burdens developers due to the large efforts in manually reviewing the false alarms reported by these tools.

Existing Techniques. As Large Language Models (LLMs) are increasingly applied to code-related tasks (Du et al., 2024a; Liu et al., 2024; Nam et al., 2024; Du et al., 2024b), researchers have leveraged their advanced program understanding capabilities to reduce false positives generated by static analyzers. However, existing LLM-based approaches have limited effectiveness in handling path feasibility analysis within large, dependency-rich codebases. Specifically, LLM4SA (Wen et al., 2024) feeds entire source-to-sink call chains along with bug reports into LLMs at once to identify false positives, without conducting fine-grained constraint analysis and relying entirely on the LLM’s ability to assess complex path feasibility. LLMDFa (Wang et al., 2024a) performs data flow analysis for each function along all potential execution paths between sources and sinks, which results in significant time and computational cost when applied to large-scale projects with complex call dependencies. While symbolic execution can perform path feasibility analysis, it suffers from path

explosion when traversing all possible paths, particularly when dealing with multiple conditional branches, loops, nested function invocations, or compound data types (Baldoni et al., 2018). Therefore, it cannot scale to large and complex software systems in practice.

Our approach. To address these limitations, we propose LLM4PFA, an LLM-powered path feasibility analysis framework designed to minimize false positives in static bug detection for complex software projects. Specifically, Given a function call trace and a target variable with its source and sink, LLM4PFA iteratively performs two-phase analysis for each function: (1) *Feasibility Constraints Extraction* first identifies critical conditional expressions impacting sink reachability; then employs LLM agents to conduct symbolic range reasoning for variables and function calls within each expression. (2) *Constraints Solving* employs constraint solver to iteratively solve the SMT query scripts generated by LLM. Overall, LLM4PFA offers the following advantages compared to traditional symbolic execution and existing LLM-based methods:

- *Precise complex constraint cascade analysis.* Breaking down inter-procedural feasibility analysis into localized symbolic range reasoning, LLM4PFA amplifies LLMs’ program comprehension strength on focused code segments (Li et al., 2023b), thereby significantly enhancing reasoning effectiveness.
- *Efficient complex context analysis.* Integrating prioritized feasible path constraints extraction method with agent-guided planning, LLM4PFA effectively mitigates path explosion in sequentially traversing deeply-nested contextual functions.

Evaluation. We manually construct a new dataset, SAFP-BENCH-C, for a more realistic evaluation, which takes approximately 100 person-hours. It contains 364 warnings generated by three advanced static analyzers, covering three important bug types (i.e., null pointer dereference, buffer overflow, and use after free) in large-scale software projects, including the Linux Kernel (with over 18 million lines of code), OpenSSL (with over 300K lines of code), and Libav (with over 600K lines of code). Our evaluation shows that LLM4PFA precisely filters out 72% to 96%) false positives reported during static bug detection, significantly outperforming all the baselines by 41.1%

- 105.7% improvements; meanwhile LLM4PFA only misses 3 real bugs of 45 true positives. Furthermore, LLM4PFA shows consistent effectiveness across four state-of-the-art LLMs, indicating the generalizability of our approach. Ablation studies further reveal that LLM4PFA outperforms two ablation variants, underscoring the key contribution of LLM agent based self-planning context analysis and iterative constraint solving proposed in our framework.

This paper makes the following contributions:

- **A novel technique** that effectively enhances complex inter-procedural feasibility analysis to minimize false positives in bug detection.
- **A new dataset** designed for a more realistic evaluation of false positive reduction, with approximately 100 person-hours dedicated to manual construction.
- **An extensive evaluation** demonstrating the effectiveness of LLM4PFA in filtering out false positives and identifying real bugs.

2 Preliminaries & Motivation

This section introduces key preliminaries of static bug detection and presents motivating examples to illustrate the false positive issues related to path feasibility validation.

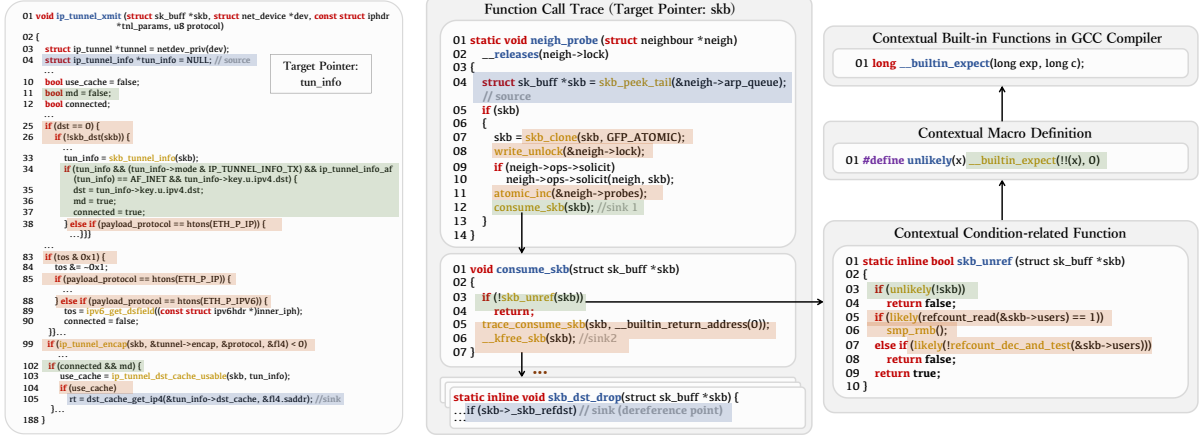
2.1 Static Bug Detection

Many bugs, such as Null Pointer Dereference (NPD) and Use After Free (UAF), follow a common pattern where a *target variable* feasibly flows from a *source* (where a potential issue originates) to a *sink* (where the issue manifests).

Source and Sink Identification. The specific definitions of the target variable, source, and sink vary by bug type. For instance, in NPD (Null Pointer Dereference) bugs, the target variable is a pointer (e.g., *tun_info* in Figure 1(a)), the source is the code expression that defines or assigns it as NULL (e.g., Line 4 in Figure 1(a)), and the sink is the code expression that dereferences the pointer (e.g., Line 105 in Figure 1(a)). For a specific bug type *bt*, given a code project $\mathcal{P}$ consisting of multiple code files and lines, the identification of the sources and sinks associated with the target variables can typically be formalized through the procedures:

$$\text{FINDSOURCE}^{bt} : \Sigma_{\mathcal{P}} \rightarrow \Sigma_{var} \times \Sigma_{expr}$$

$$\text{FINDSINK}^{bt} : \Sigma_{\mathcal{P}} \rightarrow \Sigma_{var} \times \Sigma_{expr},$$



(a) Complex Constraint Cascades

(b) Complex Constraint-Related Contextual Analysis

Figure 1: Motivating Examples

where $\Sigma_{(*)}$ represents the domain of $(*)$, i.e., Σ_{var} and Σ_{expr} denote the complete value domains of variables and code expressions, respectively. Each *source* or *sink* is represented as a pair $(var, expr)$, consisting of a target variable *var* and its corresponding code expression *expr*.

Data Flow Analysis. For each source-sink pair $(source, sink)$ sharing the same target variable *var*, data flow analysis aims to determine whether *var* can flow from *source* to *sink*. If true, a corresponding bug is reported. For instance, in Figure 1(a), data flow analysis for the pointer *tun_info* is performed between Line 4 (source) and Line 105 (sink). If a feasible execution path is found, an NPD bug is reported at the sink. This analysis can be formalized as the procedure:

TRACEFLOW : $\Sigma_{source} \times \Sigma_{sink} \rightarrow \{TRUE, FALSE\}$,

which returns *TRUE* if any feasible execution path exists between *source* and *sink*, and *FALSE* otherwise.

Path Feasibility Validation. Performing precise data flow analysis is challenging due to the complexity of validating path feasibility. Sources and sinks often reside in different functions, necessitating inter-procedural analysis to trace all potential execution paths. When the function call chain between a source and a sink is lengthy and includes multiple conditional branches, nested function invocations, or compound data types, the complexity of path feasibility validation increases exponentially (We will illustrate the challenges with examples in Section 2.2). However, traditional static analysis tools like CodeQL and Infer often incorporate over-approximation during contextual analysis, so as to

reduce the complexity of path feasibility validation. As a result, there are high false positive rates in their reported bug alarms in practice.

2.2 Motivating Examples

We present two examples to further illustrate the challenges of validating path feasibility, which lead to false positives in the NPD bug reported by CodeQL on the Linux kernel.

Challenge-1: Complex Constraint Cascades in Path Tracing. Figure 1(a) presents a false positive reported by CodeQL. In this example, the green-highlighted sections involve value assignments and constraint validation for the two *bool* variables *connected* and *md*. According to the constraint in Line 34, we can deduce that only when the target pointer *tun_info* is **not** NULL, both *connected* and *md* are *TRUE* (Lines 36 and 37). Based on the constraint in Line 102, only if both *connected* and *md* are *TRUE*, can the dereference operation (the sink) at Line 105 potentially be executed. According to both constraints, it is clear that no feasible path exists to reach the sink, and thus no NPD occurs. Beyond these green-highlighted sections, the example also involves numerous preceding and succeeding conditional constraints (highlighted in orange). Only by comprehensively cascading these constraints and analyzing the potential execution paths, can accurate path feasibility validation be achieved and false positives be eliminated. Such a high complexity is prevalent in real-world software systems, which often involve intricate function call chains and deeply nested conditional structures, making path feasibility analysis even challenging.

Challenge-2: Contextual Analysis. In addition

to the source-to-sink function call trace, precise path feasibility analysis requires reasoning numerous variable values and function call return values, further complicating the analysis process. Figure 1(b) illustrates a false positive resulting from complex contextual analysis. In Line 3 of the second function in the trace, *consume_skb* correctly performs a null check on the target pointer *skb* by invoking *skb_unref*, which returns *false* when *skb* is null. This causes the program to *return* early, preventing the null pointer from propagating to the sink. Notably, Figure 1(b) only reveals the minimal contextual information (highlighted in green) necessary for the manual inspection to identify the false positive. Existing LLM-based approaches would consider a broader set of contextual function calls (highlighted in orange), significantly increasing the complexity of context-sensitive analysis. Furthermore, due to the intricacies of the Linux kernel, this analysis incorporates not only function calls but also macro definitions and built-in GCC compiler functions, presenting additional challenges for traditional symbolic execution techniques.

3 Approach

3.1 Overview

In this work, we present an iterative path feasibility validation framework LLM4PFA to enhance complex inter-procedural feasibility analysis for minimizing false positives in static bug detection.

Figure 2 shows the overview of our approach. Given a target variable, its source and sink, and the function call trace from source to sink, LLM4PFA performs iterative feasibility analysis function-by-function. Each function is analyzed iteratively by collecting feasible path constraints and solving them to determine whether a feasible source-to-sink path exists. The process returns *True* if any feasible path exists, and *False* otherwise. This iterative process continues until either the analysis of all functions in the call trace is complete, or a *False* result is encountered.

The detailed iterative analysis within each function proceeds as follows:

- **Feasibility Constraints Extraction (Section 3.2):** Based on the Control-Flow Graph (CFG), LLM4PFA first identifies the critical path conditional branches between source and sink, and summarizes feasible path conditional expressions. Subsequently, LLM4PFA

incorporates LLM agents to conduct symbolic range reasoning for variables and function calls within each expression individually.

- **Constraints Solving (Section 3.3):** LLM4PFA leverages LLM to convert all extracted feasible path constraints into SMT query scripts, and invokes the constraint solver (i.e., Z3) for constraint solving.

3.2 Feasibility Constraints Extraction

Given a function f with the CFG entry point S_f and the exit point T_f , LLM4PFA first extracts the feasible path constraints from S_f to T_f .

To improve precision and avoid the inefficiency of exhaustively traversing all possible paths, we begin by extracting a set of critical path conditional expressions that directly impact the reachability of T_f (Section 3.2.1). Subsequently, LLM agent reasoning is employed to determine the symbolic ranges of variables and function call return values within these expressions (Section 3.2.2). These conditional expressions and symbolic ranges collectively form the feasible path constraints.

3.2.1 Critical Path Conditional Expression Extraction

Based on the extracted CFG of f , LLM4PFA first identifies the critical path conditional branches, which are a set of conditional statement nodes (i.e., branching and loop constructs) that directly impact the reachability of T_f at the control flow level. These branches are represented as:

$$N_{cs}(S_f, T_f) = N_e(T_f) \cup N_{jump}(S_f, T_f)$$

where $N_e(T_f)$ represents all conditional statement nodes nested outside the basic block containing T_f ; and $N_{jump}(S_f, T_f)$ refers to conditional statement nodes nested outside any basic block containing *Jump/Return* statements encountered between S_f and T_f . For branching statements, we focus on *If* statements (i.e., *if*, *else*, *else if* and *switch*); for *Loop* statements, we focus on *while* and *for* statements. Regarding *Jump/Return* statements, we consider all *goto* and *return* statements, as well as *break* and *continue* statements within loop constructs that are nested outside T_f .

Based on these critical path conditional branches, LLM4PFA further extracts the feasible path conditional expressions, which are represented as:

$$FPE(S_f, T_f) = \{expr_f \mid expr_f \in N_e(T_f)\}$$

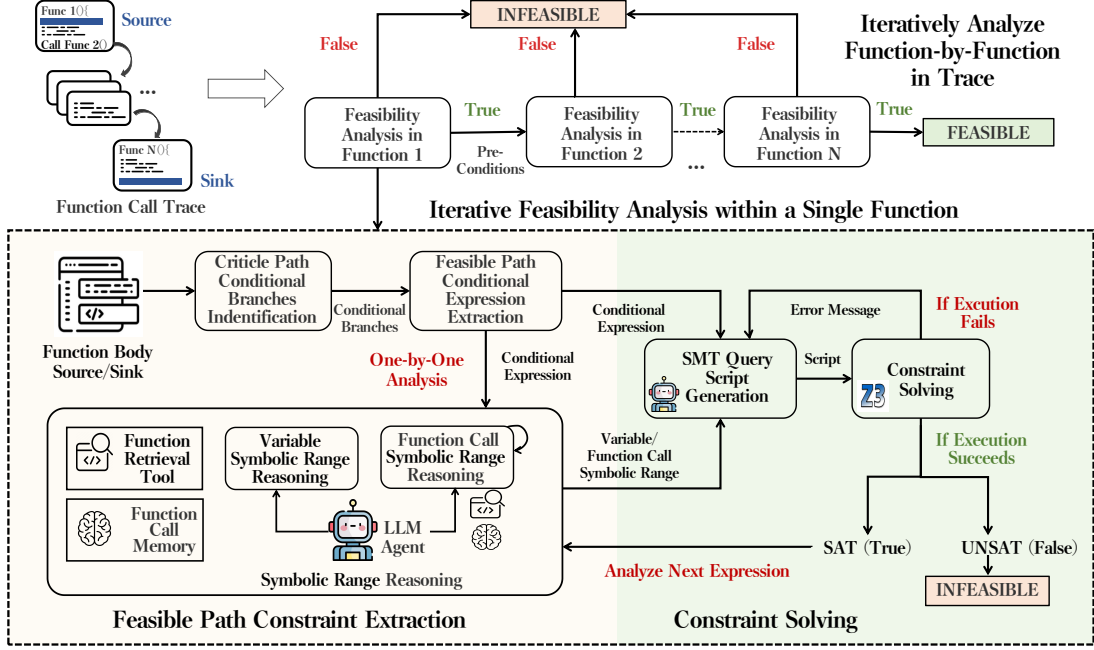


Figure 2: Overview of LLM4PFA

$$\cup \{ \neg expr_{if} \mid expr_{if} \in N_{\text{jump}}(S_f, T_f) \}$$

Where a feasible path exists only when all $expr_f$ are satisfied (i.e., the conditions for reaching the exit point branch are fulfilled), and none of $expr_{if}$ is fully satisfied (i.e., there is no early jump to other branches).

3.2.2 Context-Aware Symbolic Range Reasoning

In this step, LLM4PFA utilizes LLM agents to reason all symbolic ranges Σ_v of each feasible path conditional expression one by one. Notably, Σ_v encompasses the symbolic ranges from both variables and the function call return values in conditional expressions. These constraint-related function calls are not part of the input function call trace, and thus require additional contextual analysis.

Variable Symbolic Range Reasoning. For each variable v in a conditional expression, LLM4PFA reasons the possible symbolic ranges $\Sigma_v(P)$ of v when the conditional statement is executed. In this context, P denotes the required initial states of the program.

The initial states P are the post-conditions following the execution of S_f . These conditions encompass both the pre-conditions required by S_f and the post-conditions of the S_f statement. For the initial function in the input trace, P serves directly as the post-condition of S_f . For subsequent functions, P further incorporates all feasible path constraints associated with parameters passed to the current function, as extracted from the preced-

ing function. This ensures the propagation of constraints through function call trace, enhancing the rigor of inter-procedural analysis. The definition of P is adapted based on the specific bug type. In the case of NPD detection, the $NULL$ state of the target pointer is propagated as P throughout the entire function call trace analysis.

Inspired by chain-of-thought (CoT), LLM4PFA directs the LLM agent to perform a step-by-step simulated execution of function code, enabling more precise reasoning by LLM. Furthermore, example analyses are included within the prompt to facilitate few-shot learning for LLM agents. The detailed prompt design is in Appendix C.

Iterative constraint-related function call Symbolic range reasoning. For constraint-related function calls fc within a conditional expression, LLM4PFA aims to determine the symbolic range $\Sigma_f(P)$ of the function return value under the required initial states P . The definitions of P align with $\Sigma_v(P)$.

Figure 3 illustrates the framework for reasoning symbolic ranges of constraint-related function call return values. This reasoning process may require deep multi-layered contextual function analysis. To address the challenges posed by the complexity of context analysis (Challenge-2), we propose an LLM agent-based framework that leverages the tool-usage and self-planning capabilities of LLM agents for precise context analysis iteratively.

Given that feasible path conditional expressions may involve multiple function calls, only those

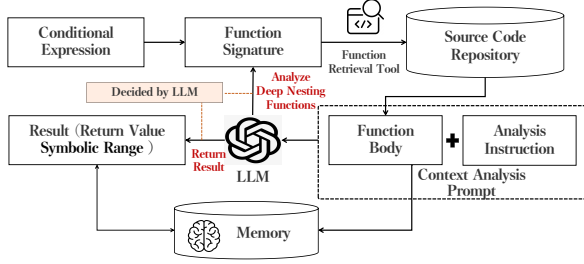


Figure 3: LLM Agent-based Context-aware Return Value Range Analysis of Function Calls

with parameters relevant to P are selected for symbolic range reasoning. For each contextual function under analysis, LLM4PFA retrieves its function body from the source code repository using a function retrieval tool. This function body, along with the initial states P , is then provided to LLMs for reasoning. During the reasoning process, the LLM agent dynamically determines whether additional deeper contextual functions are needed to refine the analysis, leveraging its self-planning capabilities. Contextual functions that are identified as affecting the return values are further analyzed with the same process. This iteration continues until the LLM agent finalizes the result or reaches the pre-defined maximum iteration limit (5 in this work).

To minimize resource consumption due to redundant analyses of identical function calls, we introduce a memory module. This module stores the return value symbolic range of previously analyzed function calls under distinct P . In subsequent analyses, results for identical scenarios can be retrieved directly from the memory module.

3.3 Constraints Solving

Given the feasible path constraints for T_f , LLM4PFA iteratively employs LLM to convert them into SMT query scripts, and invokes Z3 solver for constraint solving.

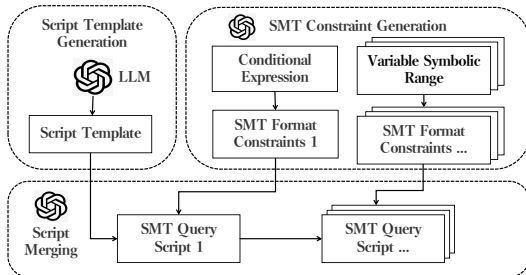


Figure 4: SMT Query Script Generation

When analyzing complex functions, LLM4PFA may solve numerous feasible path constraints. Directly converting all constraints into SMT query scripts at once can overwhelm the LLM. To ad-

dress this, we propose an iterative generation strategy. Figure 4 illustrates this generation process, which consists of three key steps:

- **Script template generation.** We utilize LLM to generate a Python template of the SMT query script, including imported packages and constraint solver definitions.
- **SMT constraints generation.** With few-shot examples, LLMs then convert path constraints (i.e., `z3.add()` format).
- **Script merging.** Lastly, LLMs merge SMT constraints with the template to create the final script, ensuring consistency between SMT variable definitions and SMT constraints.

After generating the initial script (Script 1 in Fig 4), the first round of constraint solving is performed. If the solver returns an *UNSAT* (i.e., infeasible) result, further symbolic range reasoning is terminated. During this process, each set of symbolic ranges in conditional expressions inferred by the LLM will be converted into SMT constraints and immediately merged into the script for constraint solving. Once the solver returns *UNSAT*, further reasoning is halted to avoid inefficient analysis of additional constraints.

To address script generation errors, LLM4PFA incorporates error messages from the Z3 solver into the prompt, enabling targeted fixes by the LLM. This process repeats until the Z3 solver successfully resolves the constraints or the maximum number of fix attempts (3 rounds in our setting) is reached. The detailed prompt design of SMT query script generation is in Appendix C.

4 Evaluation

4.1 Dataset

We first manually label and construct a new benchmark SAFP-BENCH-C, which includes false positive reported by state-of-the-art static analyzers on real-world complex software systems.

Bug Types. We target three representative categories of critical bugs: null pointer dereference (NPD), use-after-free (UAF), and buffer overflow (BOF). **Target Projects.** We select the latest versions of three well-maintained and large-scale C/C++ open-source projects for scanning, including the Linux kernel ([ker, 2025](#)), OpenSSL ([ope, 2025](#)), and Libav ([lib, 2025](#)). **Static Analyzers.** We use three state-of-the-art static analyzers: CodeQL ([cod, 2025](#)), Infer ([inf, 2025](#)), and CppCheck ([cpp, 2024](#)).

We adopt the following sampling strategy for manual inspection: if a static analyzer detects more than 100 warnings in a project, we randomly sample 100 instances; otherwise, all instances are included. For each warning, two participants will individually label it as True or False. In total, the construction process takes around 100 person-hours. The final benchmark SAFF-BENCH-C contains 364 warnings, of which 45 are real bugs. Detailed benchmark statistics are presented in Table 5 in Appendix A and the detailed labeling process is in Appendix A.

4.2 Method Comparison

Baselines. We compare LLM4PFA against two state-of-the-art LLM-based false positive detection methods: LLM4SA (Wen et al., 2024) and LLMDFA (Wang et al., 2024a). The detailed baseline settings are in Appendix B.

Setup and Metrics. We use three commonly used metrics in vulnerability detection tasks: accuracy, precision, and recall. Additionally, we introduce two metrics to evaluate the effectiveness in identifying false positives generated by static analyzers: False Positive Reduction Precision (FPR_P) and False Positive Reduction Recall (FPR_R), which specifically measure the precision and recall for negative samples (i.e., false positives) in the dataset. In this section, for both LLM4PFA and the baselines, we use GPT-4o-mini (gpt, 2024) as the backbone LLM. To minimize randomness, we set the temperature to 0, ensuring the LLM performs greedy decoding without sampling.

Results. Table 1 compares LLM4PFA with two baselines on SAFF-BENCH-C across three bug categories. Overall, LLM4PFA significantly outperforms all baselines across all metrics. Furthermore, LLM4PFA eliminates more false positives than the baselines (with a FPR_R improvement of 41.1% - 105.7%). Particularly, LLM4PFA precisely filters out 72% to 96% false alarms on different bugs. Meanwhile, LLM4PFA still detected 42 real bugs (true positives) out of 45, achieving a high recall rate of 0.93 (with a 3.3% - 38.8% increase). Among the three bug types, the improvement for UFA detection is limited due to the relatively simple nature of the cases, where false positives arise from significant validity checks before use or reallocation after free, which do not involve complex context or path constraints.

Table 2 presents the effectiveness of LLM4PFA

with three static analyzers in NPD detection. Among three analyzers, LLM4PFA demonstrates the greatest advantage in eliminating false positives from CodeQL (with a FPR_R improvement of 97.2% - 129.0%). This improvement is attributed to CodeQL’s relatively comprehensive null pointer check rules, which lead to more complex false positives, such as the complex constraint cascade (e.g., Example-1 in Section 2.2), and requires more extensive context analysis. Specifically, 44% of CodeQL cases involve context analysis, whereas only 0.29% and 0.36% of cases from cppcheck and infer involve context analysis, respectively.

Table 1: Comparison on Different Bugs

Type	Tech.	Acc.	FPR_P	FPR_R	Pre.	Recall
NPD	LLM4PFA	0.75	0.98	0.72	0.36	0.93
	LLM4SA	0.57	0.97	0.51	0.24	0.90
	LLMDFA	0.39	0.88	0.35	0.13	0.67
BOF	LLM4PFA	0.86	1.0	0.86	/	/
	LLM4SA	0.26	1.0	0.26	/	/
	LLMDFA	0.02	1.0	0.02	/	/
UAF	LLM4PFA	0.96	1.0	0.96	/	/
	LLM4SA	0.92	1.0	0.92	/	/
	LLMDFA	0.32	1.0	0.32	/	/

Table 2: Comparison on Different Analyzers

Analyzer	Tech.	Acc.	FPR_P	FPR_R	Pre.	Recall
CodeQL	LLM4PFA	0.75	0.97	0.71	0.42	0.91
	LLM4SA	0.45	0.93	0.36	0.24	0.88
	LLMDFA	0.38	0.85	0.31	0.17	0.73
Infer	LLM4PFA	0.68	1.0	0.63	0.30	1.0
	LLM4SA	0.59	1.0	0.53	0.25	1.0
	LLMDFA	0.27	0.71	0.26	0.07	0.33
Cppcheck	LLM4PFA	0.79	1.0	0.77	0.22	1.0
	LLM4SA	0.79	0.95	0.77	0.24	0.60
	LLMDFA	0.43	0.93	0.43	0.54	0.50

4.3 Model Versatility

To study the generality of LLM4PFA on different backbone LLMs, we build LLM4PFA with four state-of-the-art LLMs that have been widely used in bug detection, including two closed-source models, i.e., GPT-4o-mini (gpt, 2024), Claude Sonnet 3.5 (cla, 2024), and two open-source models, i.e., Qwen2.5-Coder-32B-Instruct (qwe, 2024), DeepSeek-V2-Instruct (dee, 2024). Table 3 presents the effectiveness of LLM4PFA across four different LLMs. Overall, LLM4PFA demonstrates significant effectiveness on all models (all outperforming baselines using GPT-4o-mini), showcasing the generalizability of our approach across different LLMs.

4.4 Ablation Studies

We evaluate the effectiveness of our iterative refinement-based technical design by comparing LLM4PFA with two ablation variants: NOCONTEXT performs symbolic range reasoning without

Table 3: Generality with Different LLMs

Type	Model	Acc.	FPR_P	FPR_R	Pre.	Recall
NPD	GPT-4o-mini	0.75	0.98	0.72	0.36	0.93
	Claude	0.71	0.98	0.68	0.32	0.90
	Qwen	0.65	0.96	0.62	0.27	0.83
	Deepseek	0.72	0.97	0.69	0.33	0.88
BOF	GPT-4o-mini	0.86	1.0	0.86	/	/
	Claude	0.86	1.0	0.86	/	/
	Qwen	0.86	1.0	0.86	/	/
	Deepseek	0.86	1.0	0.86	/	/
UAF	GPT-4o-mini	0.96	1.0	0.96	/	/
	Claude	0.96	1.0	0.96	/	/
	Qwen	0.96	1.0	0.96	/	/
	Deepseek	0.96	1.0	0.96	/	/

iterative context analysis driven by the LLM agent; BATCHSOL converting all feasible path constraints into a single SMT query script at once for direct constraint solving, without intermediate iterations. **Results.** Table 4 presents the comparison results of ablation variants with gpt-4o-mini. Overall, LLM4PFA demonstrates a significant superiority over NOCONTEXT and BATCHSOL in false positive elimination, showing 136.7% - 173.1% improvements in FPR_R. While BATCHSOL achieves a recall of 0.95, compared to LLM4PFA’s 0.90 in Linux kernel vulnerability detection, this discrepancy arises from a real bug being misclassified as a false alarm by LLM4PFA due to inaccurate extraction of critical path conditional expressions. This case is still considered a bug alarm in BATCHSOL due to a constraint solving failure. A high rate of constraint solving failures in BATCHSOL contributes to its low FPR_R (0.38, 57.8% less than LLM4PFA). This highlights the key contribution of LLM-based self-planning context analysis and iterative constraint solving in LLM4PFA.

Table 4: Ablation of LLM4PFA

Project	Tech.	Acc.	FPR_P	FPR_R	Pre.	Recall
Kernel	LLM4PFA	0.9	0.96	0.90	0.8	0.90
	NOCONTEXT	0.44	0.8	0.25	0.35	0.86
	BATCHSOL	0.56	0.95	0.38	0.41	0.95
Openssl	LLM4PFA	0.8	0.97	0.78	0.47	0.89
	NOCONTEXT	0.46	0.89	0.40	0.22	0.78
	BATCHSOL	0.44	0.93	0.34	0.23	0.89
Libav	LLM4PFA	0.51	1.0	0.49	0.07	1.0
	NOCONTEXT	0.21	1.0	0.18	0.04	1.0
	BATCHSOL	0.23	1.0	0.20	0.04	1.0
Total	LLM4PFA	0.75	0.97	0.71	0.42	0.91
	NOCONTEXT	0.37	0.88	0.26	0.21	0.85
	BATCHSOL	0.42	0.96	0.30	0.23	0.94

5 Related Work

Integrating LLMs with Static Analysis for Bug Detection. Currently, substantial efforts have been made to combine LLMs with static analysis to enhance detection capabilities. Li et al. (Li et al., 2023a) propose a framework that integrates UBI-Tect with LLMs to detect Use-Before-Initialization (UBI) bugs in the Linux kernel; Sun et al. (Sun et al., 2023) employ LLMs to assist static analysis

in identifying logic vulnerabilities in smart contracts. Li et al. (Li et al., 2024) propose IRIS, which systematically combines LLMs with the static analysis tool CodeQL to conduct the first whole-repository reasoning approach for comprehensive vulnerability detection. Wen et al. (Wen et al., 2024) present LLM4SA, which utilizes LLMs to process bug warning reports from various static analysis tools, leveraging domain knowledge of LLMs to reduce false positives. Wang et al. (Wang et al., 2023) propose InferROI, which employs LLMs to infer resource-oriented intentions (e.g., acquisition, release, and reachability validation) in code, combined with a two-stage static analysis approach to inspect control-flow paths for potential resource leak detection.

LLMs for Program Analysis. Shahandashti et al. (Shahandashti et al., 2024) investigate the application of LLMs in both static and dynamic program slicing, utilizing advanced prompting techniques such as few-shot learning and chain-of-thought reasoning. Wang et al. (Wang et al., 2024a) introduce LLMDFA, the first LLM-powered dataflow analysis framework, which decomposes the problem into three components: source/sink extraction, dataflow summarization, and path feasibility validation. Subsequently, they further propose LLM-SAN (Wang et al., 2024b), which leverages few-shot chain-of-thought prompting to guide LLMs in emitting data-flow paths, with validation performed through program-property decomposition.

To the best of our knowledge, no prior studies have performed the iterative decomposition of complex program analysis tasks or fine-grained symbolic range reasoning. Moreover, our approach utilizes the self-planning capabilities of LLM agents to automate the iterative analysis of deeply nested functions. This effectively mitigates the path explosion problem inherent in complex inter-procedural path feasibility analysis, which existing studies have struggled to address efficiently.

6 Conclusion

This paper proposes LLM4PFA, an iterative path feasibility analysis framework powered by LLM agents, which effectively enhances complex inter-procedural feasibility analysis for minimizing false positives in bug detection. Our evaluation shows that LLM4PFA substantially outperforms all baselines in both bug detection (with 0.93 recall) and false positive elimination.

References

2024. [Claude sonnet 3.5](#).
2024. [Cppcheck](#).
2024. [Deepseek-coder-v2-instruct](#).
2024. [Gpt-4-turbo](#).
2024. [Qwen2.5-coder-32b- 256 instruct](#).
2025. [Codeql](#).
2025. [Infer](#).
2025. [Libav](#).
2025. [Linux-kernel 6.9.6](#).
2025. [Openssl](#).
- Roberto Baldoni, Emilio Coppa, Daniele Cono D’elia, Camil Demetrescu, and Irene Finocchi. 2018. [A survey of symbolic execution techniques](#). *ACM Comput. Surv.*, 51(3).
- Yuandao Cai, Peisen Yao, Chengfeng Ye, and Charles Zhang. 2023. Place your locks well: understanding and detecting lock misuse bugs. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3727–3744.
- Yuandao Cai, Peisen Yao, and Charles Zhang. 2021. Canary: practical static detection of inter-thread value-flow bugs. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 1126–1140.
- Yuandao Cai, Chengfeng Ye, Qingkai Shi, and Charles Zhang. 2022. Peahen: Fast and precise static deadlock detection via context reduction. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 784–796.
- Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024a. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13.
- Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2024b. [Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag](#).
- Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2023a. [The hitchhiker’s guide to program analysis: A journey with large language models](#).
- Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. 2023b. Loogle: Can long-context language models understand long contexts? *arXiv preprint arXiv:2311.04939*.
- Ziyang Li, Saikat Dutta, and Mayur Naik. 2024. Llm-assisted static analysis for detecting security vulnerabilities. *arXiv preprint arXiv:2405.17238*.
- Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977*.
- Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13.
- Kimya Khakzad Shahandashti, Mohammad Mahdi Mohajer, Alvine Boaye Belle, Song Wang, and Hadi Hemmati. 2024. Program slicing in the era of large language models. *arXiv preprint arXiv:2409.12369*.
- Qingkai Shi, Rongxin Wu, Gang Fan, and Charles Zhang. 2020. Conquering the extensional scalability problem for value-flow analysis frameworks. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pages 812–823.
- Qingkai Shi, Xiao Xiao, Rongxin Wu, Jinguo Zhou, Gang Fan, and Charles Zhang. 2018. Pinpoint: Fast and precise sparse value flow analysis for million lines of code. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 693–706.
- Yulei Sui, Ding Ye, and Jingling Xue. 2012. Static memory leak detection using full-sparse value-flow analysis. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis*, pages 254–264.
- Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2023. [When gpt meets program analysis: Towards intelligent detection of smart contract logic vulnerabilities in gptscan](#).
- Carmine Vassallo, Sebastiano Panichella, Fabio Palomba, Sebastian Proksch, Harald C Gall, and Andy Zaidman. 2020. How developers engage with static analysis tools in different contexts. *Empirical Software Engineering*, 25:1419–1457.
- Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. 2024a. Llmdfa: Analyzing dataflow in code with large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, and Xiangyu Zhang. 2024b. Sanitizing large language models in bug detection with data-flow. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3790–3805.

Chong Wang, Jianan Liu, Xin Peng, Yang Liu, and Yiling Lou. 2023. Boosting static resource leak detection via llm-based resource-oriented intention inference. *arXiv preprint arXiv:2311.04448*.

Cheng Wen, Yuandao Cai, Bin Zhang, Jie Su, Zhiwu Xu, Dugang Liu, Shengchao Qin, Zhong Ming, and Tian Cong. 2024. Automatically inspecting thousands of static bug warnings with large language model: How far are we? *ACM Transactions on Knowledge Discovery from Data*, 18(7):1–34.

Hua Yan, Yulei Sui, Shiping Chen, and Jingling Xue. 2018. Spatio-temporal context reduction: A pointer-analysis-based static approach for detecting use-after-free vulnerabilities. In *Proceedings of the 40th International Conference on Software Engineering*, pages 327–337.

A Dataset

SAFP-BENCH-C contains 364 warnings generated by three static analyzers scanning three C/C++ projects, of which 45 are true bugs. Table 5 presents the detailed statistics of SAFP-BENCH-C.

Table 5: Statistics of SAFP-BENCH-C

Type	Analyzer	Kernel	OpenSSL	Libav	Total
NPD	CodeQL	70 (22)	50 (8)	57 (2)	177 (32)
	CppCheck	83 (8)	5 (0)	10 (2)	98 (10)
	Infer	/	16 (3)	6 (0)	24 (3)
UAF	CodeQL	16 (0)	9 (0)	/	25 (0)
BOF	CodeQL	/	42 (0)	/	42 (0)

The identifiers used for each static analyzer across three CWEs are provided in Table 6. Infer and CPPcheck did not detect any warnings for BOF and UAF in the target projects.

Table 6: Analyzer identifiers across CWEs

Type	Analyzer	Command
NPD	CodeQL	MissingNullTest.q1;
	CppCheck	nullPointer;
	Infer	Null Dereference;
UAF	CodeQL	UseAfterFree.q1
BOF	CodeQL	OverflowBuffer.q1

Labeling process. For each warning, two participants will individually label it as True or False; for the inconsistent cases, a third judge will be introduced to solve the conflicts. If the participants cannot reach agreement after discussion, the warning would be filtered out. All the participants have over five-year experience on C/C++; and each would received 200 dollars for the labeling. The experiments are approved by Ethics Review Board.

B Baseline Settings

In RQ1, We compare LLM4PFA against two baselines: LLM4SA (Wen et al., 2024) and

LLMDFA (Wang et al., 2024a). For LLM4SA, we use its public implementation with the following prompt:

You are an expert C/C++ programmer.

Task Description:

The code snippet and the bug report will be provided to you for the purpose of examining the presence of the bug within the code snippet. Initially, you need to explain the behavior of the code. Subsequently, you can determine whether the bug is a true positive or a false positive based on the explanation. To conclude your answer, you will provide one of the following labels: @@@ real bug @@@, @@@ false alarm @@@, or @@@ unknown @@@.

Bug Report: [Bug Report]

Code Snippet: [Code Snippet]

For LLMDFA, we omit the Source/Sink Extraction phase and instead use the sources and sinks from SAFP-BENCH-C for Dataflow Summarization and Path Feasibility Validation. The prompt template in NPD detection is as follows.

Prompt for Dataflow Summarization:

System Role: You are a C programmer and very good at analyzing C code.

Task Description: Determine whether two pointer variables at two lines have the same value (i.e., both pointing to the same address or both NULL).

Analysis Rules:

If they are the same pointer variable and not overwritten between two lines, the answer should be yes.

If the pointer variable a is assigned to the variable b, and there is no overwrite upon them between two lines, the answer should be yes.

[Other Rules]

Examples:

Example 1: User: [Program] [Question]

System: [Answer: Yes][Explanation: y is assigned with x at line 2 and not overwritten between line 2 and line 3, so the value of y before line 3 and x after line 1 are the same]

[Other Examples]

Question: Now I give you a function: [FUNCTION]

Please answer: Dose [VAR1] used at line [LINE1] have the same value as [VAR2] defined at line [LINE2]

Please think it step by step. Return Yes/No with the explanation.

Prompt for Dataflow Validation:

System Role: You are a good programmer and familiar with Z3 Python binding.

Description: Please write a Python program using Z3 Python binding to encode the path condition.

Path Info: Here is the path: [path]. Note that the value of [variable] is 0.

The branch information is as follows: [branch info].

The early jump information is as follows: [early jump info].

Synthesis Task: Please write a Python script to solve the path condition using Z3 Python binding. You can refer to the skeleton: [skeleton]

Fixing Task: Here is the synthesized result of the last round: [script]. When executing the synthesized

script, we encounter the following error: [error message]. Please fix the bug and return a runnable script.

C Prompt Design of LLM4PFA

C.1 Prompt in Context-Aware Symbolic Range Reasoning

Prompt for variable symbolic range reasoning:

System Role: You are an expert C/C++ programmer.

Task Description: Your goal is to determine the possible value ranges for each variable in [conditional statement] assuming the [P] when [conditional statement] is executed.

Important:

- Analyze each line sequentially through symbolic execution.
- Consider function semantics based on their names.
- Pay special attention to pointer operations and conditions.
- Track how variable ranges change through the execution path.

After the analysis, conclude your answer.

Code Snippet: [Code Snippet]

Please provide your analysis in clear steps, explaining how you arrived at the value ranges.

Prompt for contextual analysis:

System Role: You are an expert C/C++ programmer.

Task Description: Given the function [function name] with body [function body], your goal is to determine the possible value ranges for the function return value when [P]. If the value range cannot be directly determined and further investigation in deeper function is required, please use the search_context tool to retrieve the function body for further analysis.

Important:

...

Code Snippet: [Code Snippet]

Please provide your analysis in clear steps, explaining how you arrived at the value ranges.

C.2 Prompt in SMT Query Script Generation

Prompt in SMT constraints generation:

System Role: You are an SMT constraint expert.

Task Description: Convert the following C/C++ conditional expressions into Z3 SMT solver constraints: [Conditional Expressions]

When generating Z3 SMT solver scripts, follow these critical guidelines:

[SMT Query Script Generation Rules]

Return only the executable Z3 constraints in the following format:

```
```python
var_name1 = Int('var 1')
var_name2 = Bool('var 2')
s.add(constraint 1)
s.add(constraint 2)
...
```
```

Prompt in script merging:

System Role: You are an expert Z3 constraint solver.

Task Description: I'll provide a Z3 constraint script and a set of additional constraints. Please combine them into a single script and keeping all existing constraints (both from the original script and the new constraint sets). If any new constraints are missing variable definitions, make sure to add or modify the necessary declarations. Return your solution as a function named *check_constraints()* that returns a boolean indicating whether the constraints are satisfiable. The function should be ready to execute and no imports or extra content needed.

Original Z3 script: [SMT Query Script]

Additional Constraint Set: [SMT Format Constraints]

Prompt in script fix:

Task Description: The Z3 script you generated contains errors. Please fix and regenerate the Z3 Python script. The script should be directly executable.

Error details: [Error Message]

Additional Requirements: [SMT Query Script Generation Rules]